

Sri Lanka Institute of Information Technology

B.Sc. (Hons) Information Technology



Specialization in Information Technology

IT3020 - Database Systems

Laboratory Worksheet 08

Student ID : IT23421226

Contents

PART 1 — Create the Database and Table.....	3
PART 2 — Write the Transaction Script (Lab8.sql)	4
Simulation 1: Atomicity.....	4
SIMULATION 2 — Concurrency Control.....	6
SIMULATION 3 — Deadlock	10

PART 1 — Create the Database and Table

Step 1 — Create the Bank Database

Step 2 — Create the Account Table (with constraint balance >= 0)

Step 3 — Insert the Two Accounts (each Rs.100)

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the Object Explorer displays the server structure, including the 'BankDB' database. The main pane shows the SQL Query window with the following script:

```
1 CREATE DATABASE BankDB;
2
3 USE BankDB;
4
5 CREATE TABLE Account
6 (
7     AccountNo VARCHAR(20) PRIMARY KEY,
8     AccountType VARCHAR(20),
9     CustomerName VARCHAR(50),
10    Balance DECIMAL(10,2) CHECK (Balance >= 0)
11 );
12
13 INSERT INTO Account VALUES ('1234567899','Savings','Same Customer',100.00);
14 INSERT INTO Account VALUES ('1234567898','Savings','Same Customer',100.00);
15
16 SELECT * FROM Account;
```

Below the script, the status bar indicates '68 %' and 'No issues found'. The 'Results' tab is active, displaying the following data:

	AccountNo	AccountType	CustomerName	Balance
1	1234567898	Savings	Same Customer	100.00
2	1234567899	Savings	Same Customer	100.00

PART 2 — Write the Transaction Script (Lab8.sql)

```
Lab8.sql - ...G\USER (54))* X
13 INSERT INTO Account VALUES ('1234567899', 'Savings', 'Same Customer', 100.00);
14
15
16 SELECT * FROM Account;
17
18 SET XACT_ABORT ON;
19
20 DECLARE @sourceAcc VARCHAR(20) = '1234567899';
21 DECLARE @destAcc VARCHAR(20) = '1234567898';
22 DECLARE @fundsToTransfer DECIMAL(10,2) = 150.00;
23
24 BEGIN TRANSACTION;
25
26 -- Step 1: Credit destination account first
27 UPDATE Account
28 SET Balance = Balance + @fundsToTransfer
29 WHERE AccountNo = @destAcc;
30
31 -- Step 2: Deduct from source account
32 UPDATE Account
33 SET Balance = Balance - @fundsToTransfer
34 WHERE AccountNo = @sourceAcc;
35
36 COMMIT TRANSACTION;
37
```

68 % No issues found Ln: 18, Ch: 1 (473 chars, 19 lines) SPC

Messages

(1 row affected)
Msg 547, Level 16, State 0, Line 32
The UPDATE statement conflicted with the CHECK constraint "CK_Account__Balance__49C3F6B7". The conflict occurred in database "BankDB", table "dbo.Account", column 'Balance'.
Completion time: 2026-03-26T21:18:13.9490781+05:30

SIMULATION 1: Atomicity

Input: @sourceAcc = 1234567899
@destAcc = 1234567898
@ fundsToTransfer = 150.00

Step 4 — Run the Script

Step 5 — Check balances again

Lab8.sql - ...G\USER (54))*

33
34
35
36
37
38
39

```
SET balance = balance - @transferAmount
WHERE AccountNo = @sourceAcc;

COMMIT TRANSACTION;

--check balance again
SELECT * FROM Account;
```

68 %

✔ No issues found

Results

Messages

	AccountNo	AccountType	CustomerName	Balance
1	1234567898	Savings	Same Customer	100.00
2	1234567899	Savings	Same Customer	100.00

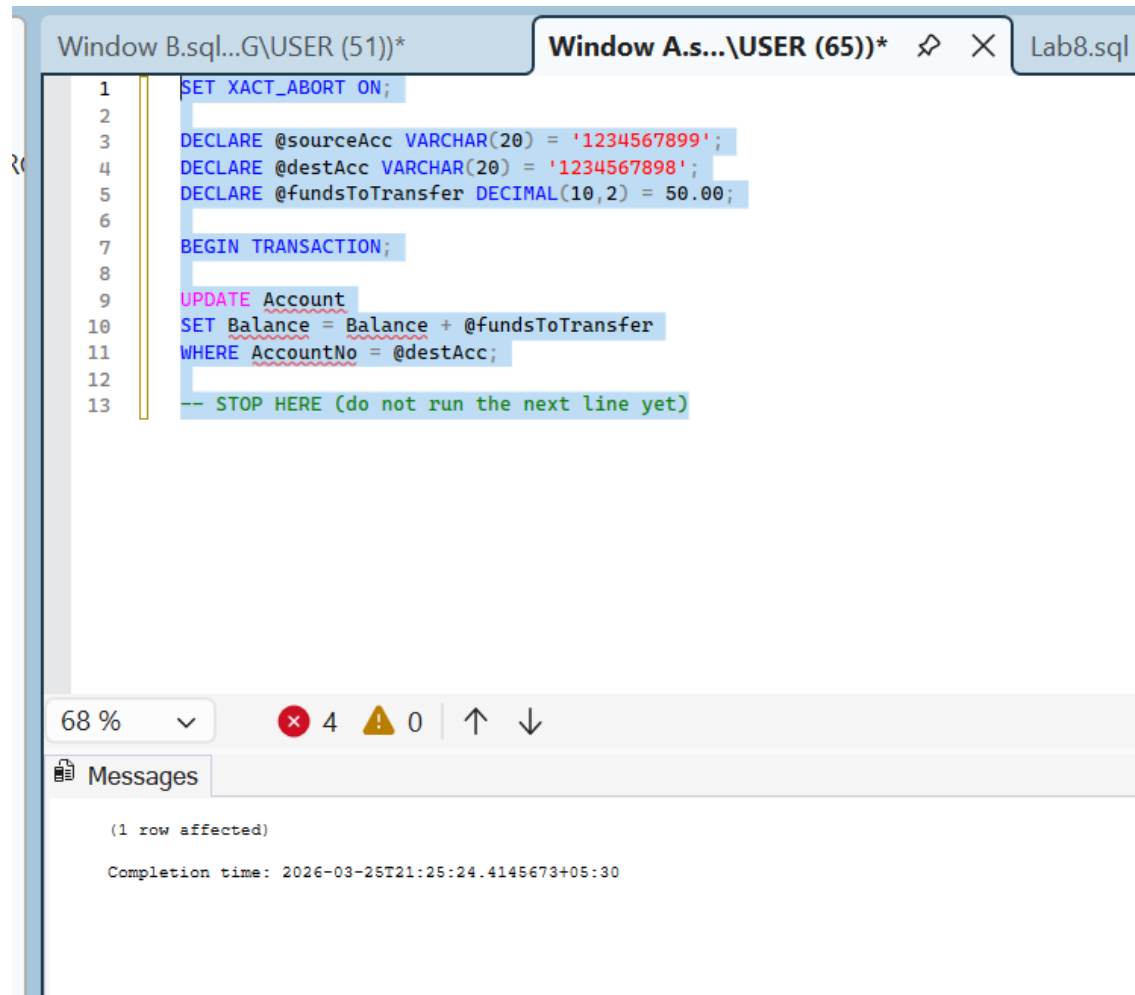
Simulation 1 Explanation

This simulation demonstrates the Atomicity property of transactions. The transfer amount (Rs.150) is greater than the balance in the source account (Rs.100). When SQL Server tries to deduct the amount, the CHECK constraint ($\text{Balance} \geq 0$) is violated. Because XACT_ABORT is set to ON, the entire transaction is automatically rolled back. Therefore, both accounts remain unchanged. This shows that a transaction is either fully completed or not executed at all.

SIMULATION 2 — Concurrency Control

Step 6 —modified script in both windows

Step 7 — Run Window A first



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a script window titled 'Window B.sql...G\USER (51))*' containing the following T-SQL code:

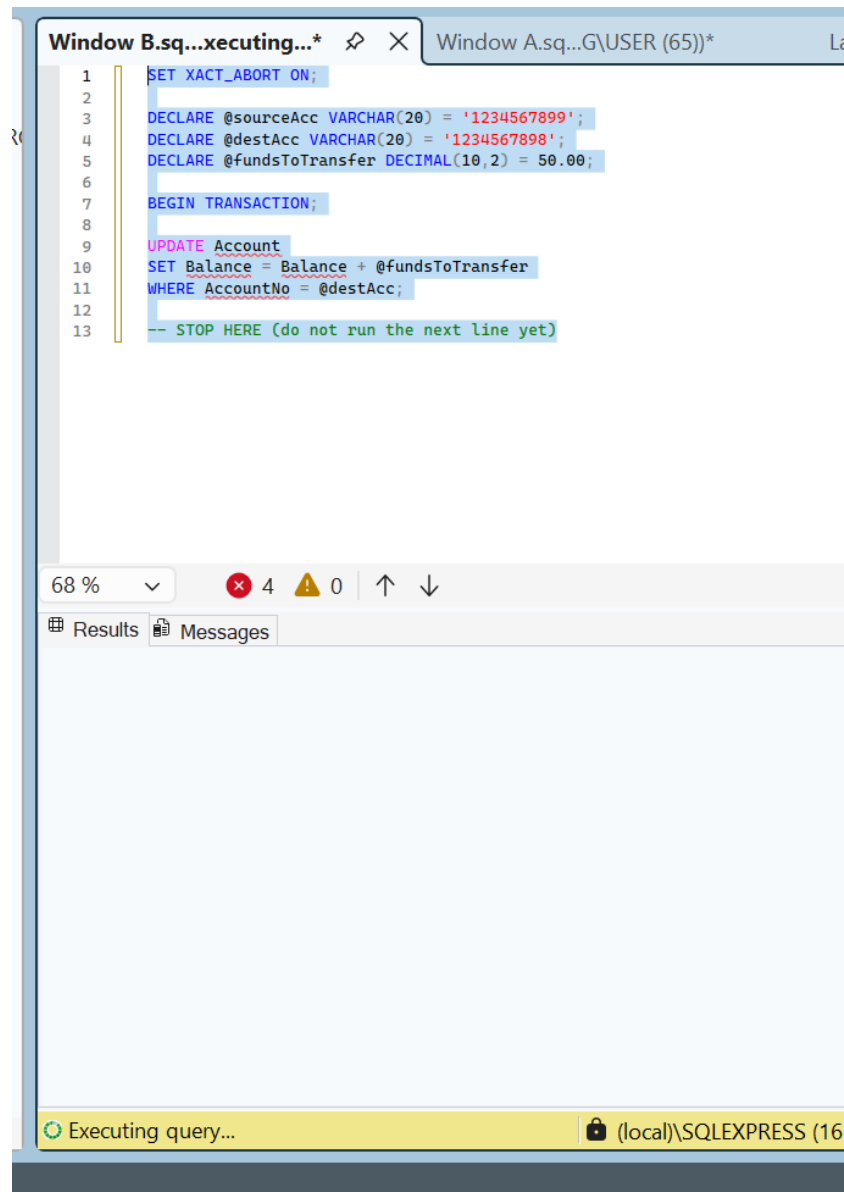
```
1 SET XACT_ABORT ON;  
2  
3 DECLARE @sourceAcc VARCHAR(20) = '1234567899';  
4 DECLARE @destAcc VARCHAR(20) = '1234567898';  
5 DECLARE @fundsToTransfer DECIMAL(10,2) = 50.00;  
6  
7 BEGIN TRANSACTION;  
8  
9 UPDATE Account  
10 SET Balance = Balance + @fundsToTransfer  
11 WHERE AccountNo = @destAcc;  
12  
13 -- STOP HERE (do not run the next line yet)
```

The bottom pane shows the execution results for the script. The status bar indicates '68 %' completion, with 4 errors (red X icon) and 0 warnings (yellow triangle icon). The 'Messages' tab is active, displaying the following output:

```
(1 row affected)  
  
Completion time: 2026-03-25T21:25:24.4145673+05:30
```

Step 8 — Run Window B

It will NOT finish



Step 9 — Next finish Window A

The screenshot displays the SQL Server Enterprise Manager interface. At the top, two window tabs are visible: "Window B.sql... Executing...*" and "Window A.s...\(USER (51))*". The active window, "Window A", contains a SQL script with the following lines:

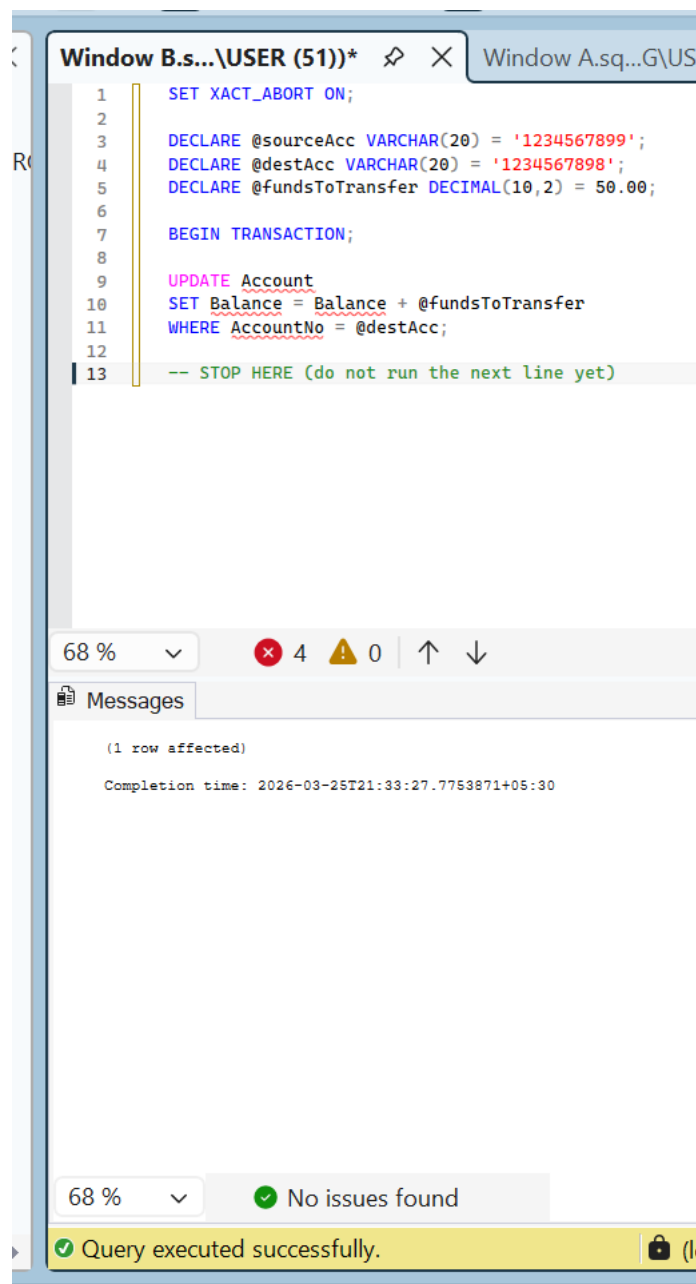
```
1 SET XACT_ABORT ON;
2
3 DECLARE @sourceAcc VARCHAR(20) = '1234567899';
4 DECLARE @destAcc VARCHAR(20) = '1234567898';
5 DECLARE @fundsToTransfer DECIMAL(10,2) = 50.00;
6
7 BEGIN TRANSACTION;
8
9 UPDATE Account
10 SET Balance = Balance - @fundsToTransfer
11 WHERE AccountNo = @sourceAcc;
12
13 COMMIT TRANSACTION;
14
15 -- STOP HERE (do not run the next line yet)
```

Below the script editor, a status bar shows "68 %" zoom, 4 errors (red X icon), and 0 warnings (yellow triangle icon). A "Messages" tab is active, displaying the following information:

(1 row affected)
Completion time: 2026-03-25T22:03:12.0637778+05:30

At the bottom of the window, another status bar shows "68 %" zoom, a green checkmark icon, and the text "No issues found". The very bottom of the interface features a yellow status bar with a green checkmark icon, the text "Query executed successfully.", and the server name "(local)\SQLEXPRESS (16.0)".

Now Window B will continue automatically.



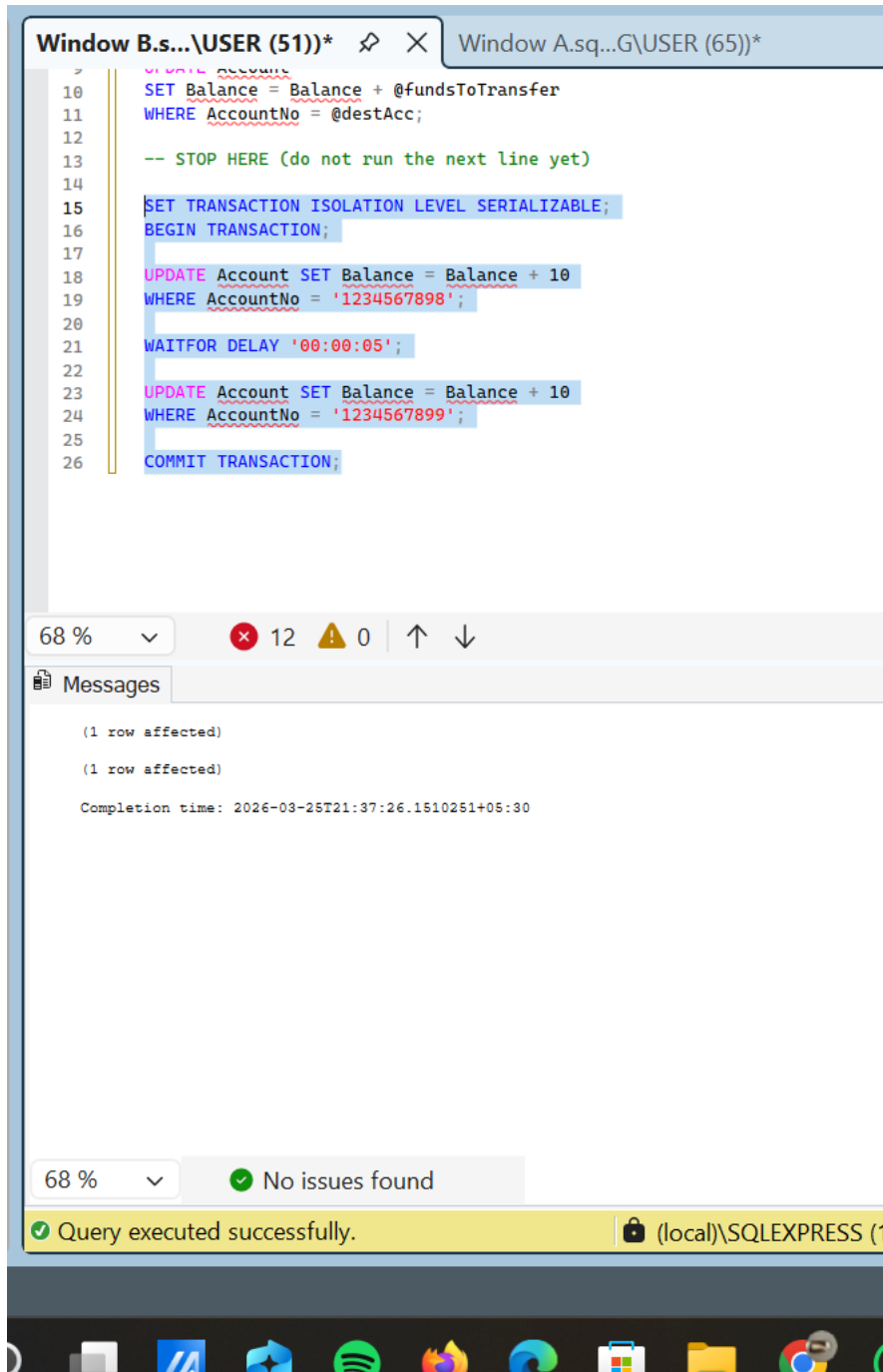
Simulation 2 Explanation

This simulation demonstrates concurrency control using locks. When the first transaction (Connection A) updates the account, SQL Server locks that row. When the second transaction (Connection B) tries to update the same row, it must wait until the lock is released. Once transaction A completes and commits, the lock is released and transaction B continues. This shows how SQL Server maintains data consistency when multiple users access the same data simultaneously.

SIMULATION 3 — Deadlock

Step 10 — Run both quickly

One of them will fail with this error (Window A shows this error)



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a SQL script in a window titled 'Window A.s... \USER (65))'. The script contains the following T-SQL code:

```
13 COMMIT TRANSACTION;
14 -- STOP HERE (do not run the next line yet)
15
16 SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
17 BEGIN TRANSACTION;
18
19 UPDATE Account SET Balance = Balance + 10
20 WHERE AccountNo = '1234567899';
21
22 WAITFOR DELAY '00:00:05';
23
24 UPDATE Account SET Balance = Balance + 10
25 WHERE AccountNo = '1234567898';
26
27 COMMIT TRANSACTION;
28
29
```

The bottom pane shows the 'Messages' window with the following error message:

```
(1 row affected)
Msg 1205, Level 13, State 51, Line 24
Transaction (Process ID 65) was deadlocked on lock resources with another process and has been chosen as the deadlock victim. Rerun the transaction.
Completion time: 2026-03-25T21:37:26.1326999+05:30
```

Simulation 3 Explanation

This simulation demonstrates a deadlock situation. Transaction A locks account 1234567899 and then tries to access account 1234567898. At the same time, Transaction B locks account 1234567898 and then tries to access account 1234567899. Since both transactions are waiting for each other to release locks, a deadlock occurs. SQL Server automatically selects one transaction as the deadlock victim and cancels it to allow the other transaction to continue.